
HR Assist

Agentic HR Chat Assistant — Architecture & Design

A LangGraph.js agent on Claude Haiku 4.5 that answers employees' leave and HR-policy questions, cites the policy documents, does leave arithmetic in deterministic code, applies for and cancels leave, and lets managers approve their team's requests — always asking the user to confirm before it writes anything. All data in this project is synthetic.

Stack	TypeScript · LangGraph.js · LangChain (Anthropic) · Express · SQLite · React + MUI · Vite
Repository	github.com/Muthukumar-ramasamy/HR-Chat-Agent
Date	October 2026

Contents

1. What it can do
2. Screens
3. Quick start
4. Architecture
5. Design decisions
6. Security and data
7. Testing
8. Project structure
9. Limitations and future enhancements

HR Assist answers employees' leave and HR-policy questions, files and cancels leave requests for them, and lets managers review and approve their team's leave. It is a **LangGraph.js agent** running on **Claude Haiku 4.5**. The agent looks up the user's own records, searches the policy documents (citing them), does leave arithmetic in deterministic code, and asks the user to confirm before it writes anything.

All data in this project is **synthetic**: fictional company (Acme Corp), fictional employees, sample policies. No real HR or personal data is used.

Stack: TypeScript end to end · LangGraph.js · LangChain (Anthropic) · Express · SQLite (better-sqlite3) · React 19 + MUI · Vite · LangSmith tracing

 **Architecture & design document (PDF):** <docs/HR-Assist-Architecture.pdf>

1 What it can do

Ask	What the agent does
"What's my leave balance?"	One tool call → balances per leave type
"Can I take Dec 22 to Jan 2 off?"	Resolves the year, counts working days (skips weekends and the office's holidays), checks the balance and the leave type's policy rules, reports loss of pay if any
"How many earned leave days can I carry forward?"	Searches the policy documents and answers with a citation, e.g. (<i>Leave Policy §4</i>)
"What is the bereavement leave policy?"	Says the policy documents don't cover it instead of guessing
"What about sick leave instead?"	Uses conversation memory for the follow-up
"Apply for earned leave from Dec 22 to Jan 2"	Validates the request in code, pauses for confirmation , writes it only after the user clicks Submit
"Cancel my leave on Oct 16"	Finds the request, checks it can still be cancelled, pauses for confirmation , then cancels it and returns the days to the balance
"What is Dev Patel's leave balance?"	Refuses: users can only see their own data
<i>Manager</i> : "Any leave requests waiting for my approval?"	Lists direct reports' pending requests
<i>Manager</i> : "Approve request #4"	Pauses for confirmation , then approves (or rejects) — only for direct reports
<i>Manager</i> : "Who on my team is on leave next month?"	Team leave calendar for direct reports

2 Screens

Captured from the recorded demo (synthetic data). The caption under each reply shows the tools the agent used, the number of model calls and the tokens spent.

HR Assist

Asha Rao · G5 · Chennai

The policy for leave calculation is as follows: Working days (weekends + Chennai holidays excluded) computed in code, plus the leave type's policy rules.

Tools: search_policy · 2 model calls · 4.9k tokens

What's my leave balance?

Leave Type	Available	Allocated	Carried Forward	Used	Pending
Casual (CL)	10	12	0	1	1
Earned (EL)	21	18	8	5	0
Sick (SL)	14	12	4	2	0
Total	45	—	—	—	—

Tools: get_leave_balance · 2 model calls · 5.1k tokens

Can I take Dec 22 to Jan 2 off?

Yes, you can take Dec 22, 2026 to Jan 2, 2027 off. That's **7 working days** (Christmas and New Year's Day don't count). You have 21 EL available, so you'd have 14 remaining. Note: requests of 3+ days need 7 days' advance notice (Leave Policy §4).

Tools: calculate_leave, search_policy · 2 model calls · 5.7k tokens

Figure A. Balance (one tool call) and “Can I take Dec 22 to Jan 2 off?”: 7 working days computed in code, plus the earned-leave notice rule with citation

HR Assist

Asha Rao · G5 · Chennai

5 · Human in the loop Nothing is saved yet. Input is disabled until the user decides.

instead?

Yes, you can take Dec 22, 2026 to Jan 2, 2027 off as sick leave. That's **7 working days**, and you have 14 SL available, leaving 7 remaining. However, since it's more than 2 consecutive days, you'll need a **medical certificate** (Leave Policy §3 Sick Leave). The 7-day advance notice still applies for requests of 3+ days.

Tools: calculate_leave, search_policy · 2 model calls · 6.1k tokens

Apply for earned leave from Dec 22 to Jan 2 for a family trip

Confirm leave request

Leave type

Earned Leave (EL)

From

Tue, 22 Dec 2026

To

Sat, 2 Jan 2027

Working days

7

Balance

21 → 14 days

Reason

family trip

The request goes to your manager for approval after you submit.

SUBMIT

CANCEL

Figure B. Follow-up grounded with fresh tool calls, then a leave application paused for confirmation (LangGraph interrupt)

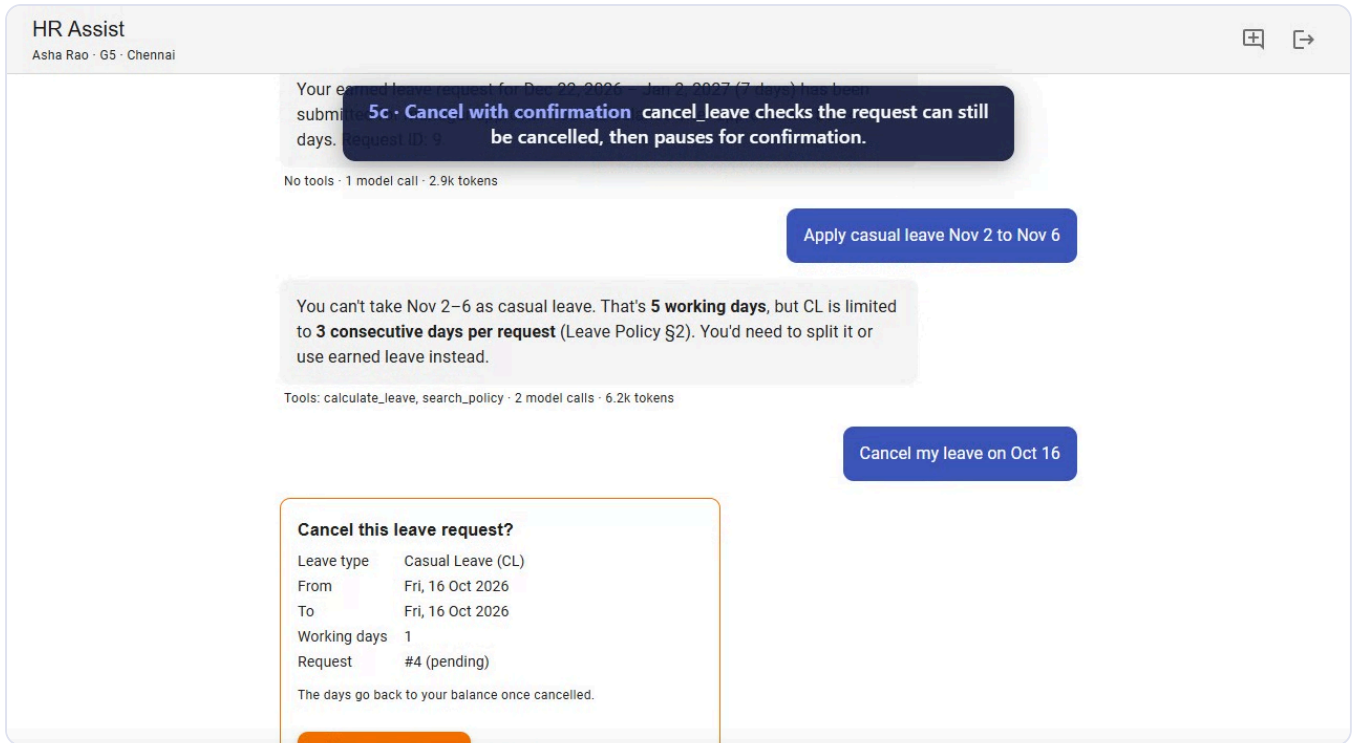


Figure C. A rule enforced in code (casual leave ≤ 3 days), then cancelling a request with its own confirmation card

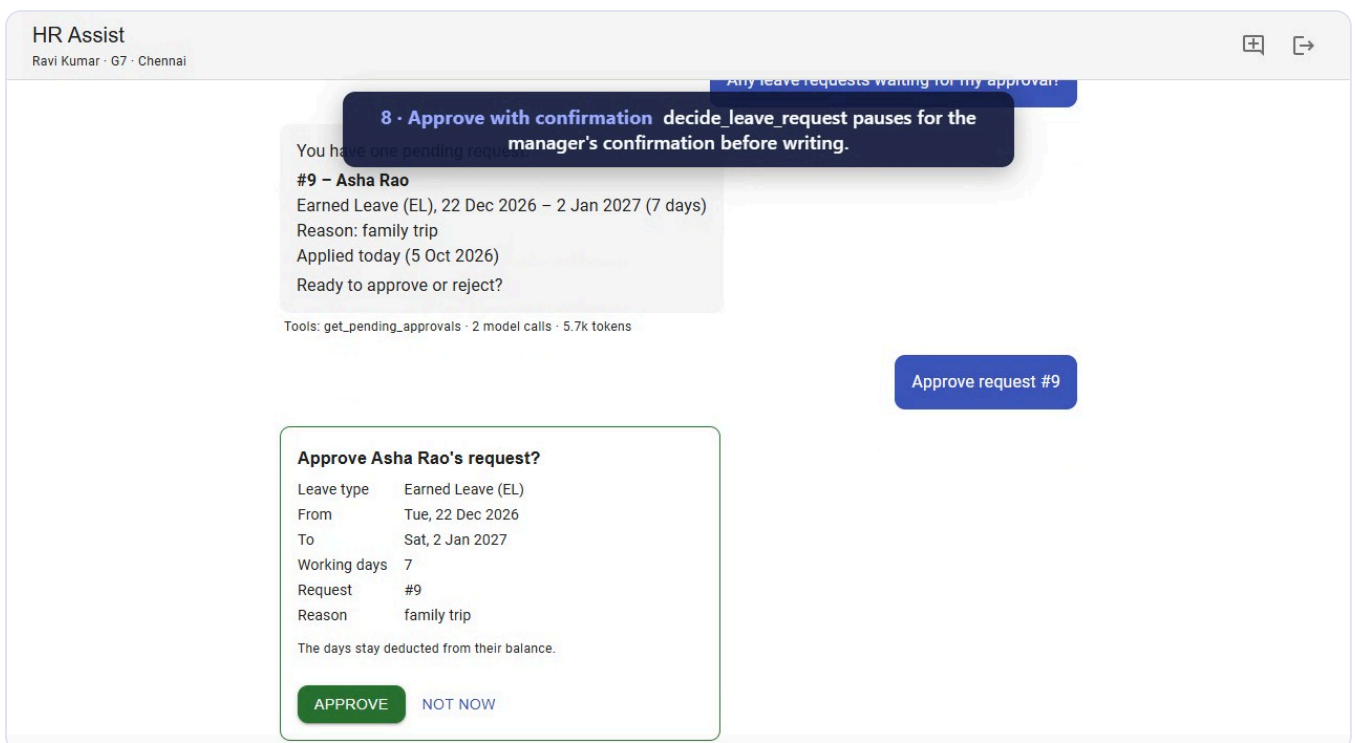


Figure D. Manager view: pending approvals for direct reports and the approve confirmation card

3 Quick start

Prerequisites: Node.js 22.12 or newer (developed on Node 25), an [Anthropic API key](#).

```
npm run setup # installs root, server and client dependencies; seeds the
database
cp server/.env.example server/.env # Windows: copy server\.env.example server\.env
```

Edit `server/.env` :

Variable	Required	Notes
<code>ANTHROPIC_API_KEY</code>	yes	Claude key
<code>JWT_SECRET</code>	yes	32+ random characters: <code>node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"</code>
<code>LLM_PROVIDER</code> / <code>LLM_MODEL</code>	no	Defaults to <code>anthropic</code> / <code>claude-haiku-4-5</code> . <code>google</code> + <code>GOOGLE_API_KEY</code> switches to Gemini
<code>LANGSMITH_TRACING</code> , <code>LANGSMITH_API_KEY</code> , <code>LANGSMITH_PROJECT</code>	no	Turns on LangSmith traces of every model and tool call
<code>APP_TODAY</code>	no	Pin "today" (YYYY-MM-DD) for reproducible demos

```
npm run dev # API on :3001 and UI on :5173 together
```

Open <http://localhost:5173> and pick a demo account (password for all: `Password@123`) :

Account	Why it's interesting
<code>asha.rao@example.com</code>	Main demo user (Chennai): leave history, a pending request
<code>dev.patel@example.com</code>	Bengaluru office: different regional holidays
<code>priya.nair@example.com</code>	Joined Jul 2026: pro-rated quota, not yet eligible for earned leave
<code>ravi.kumar@example.com</code>	Manager of the other three: team leave view and approvals

Other scripts (from the project root): `npm test` (unit/integration tests, no LLM calls), `npm run eval` (evaluation against the real model), `npm run typecheck` , `npm run db:reset` (restore the demo data). In `server/` : `npm run chat` (terminal chat; `-- --as E1000` to switch user), `npm run graph` (print the agent graph as Mermaid). `npm run dev` first stops leftover dev servers of this project still holding ports 3001/5173.

4 Architecture

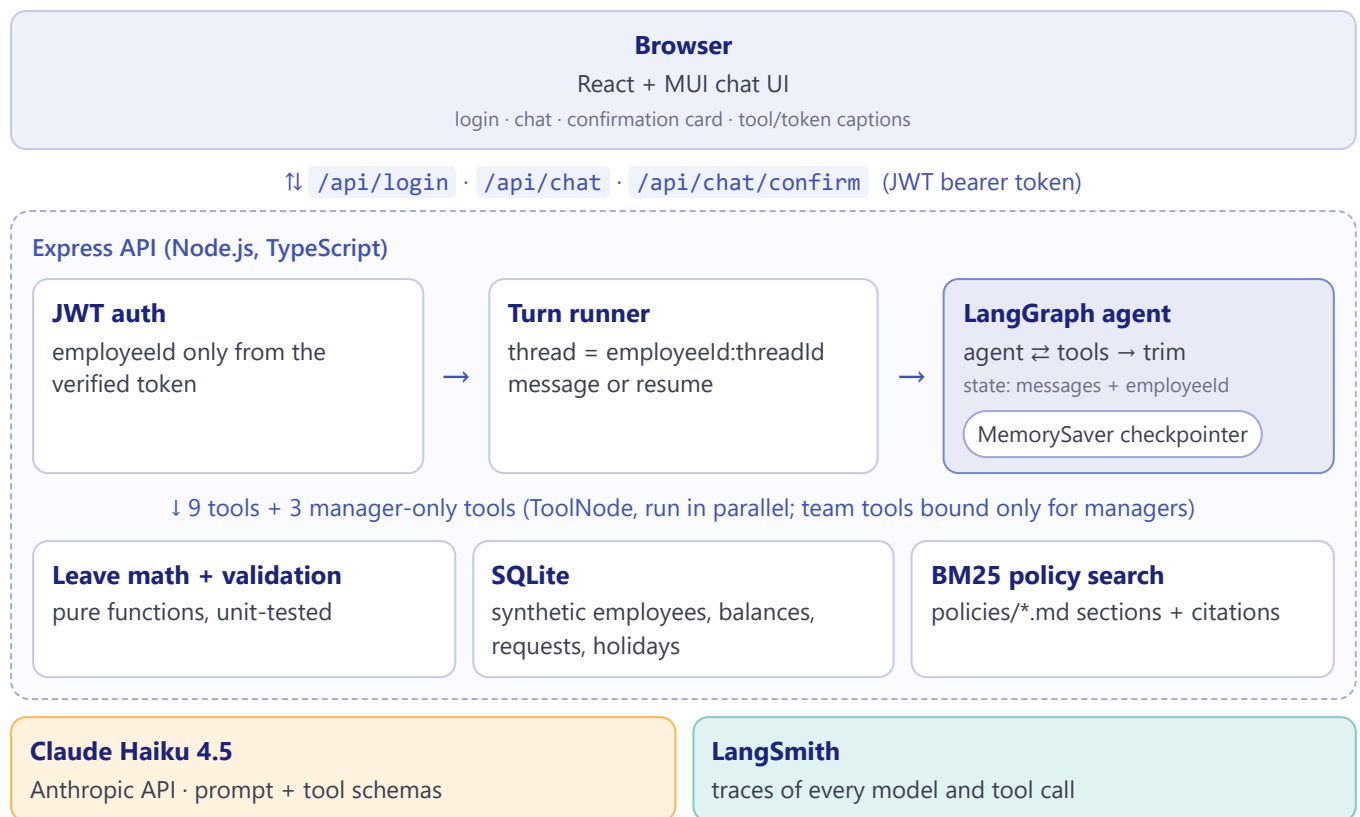


Figure 1. System architecture

The agent graph

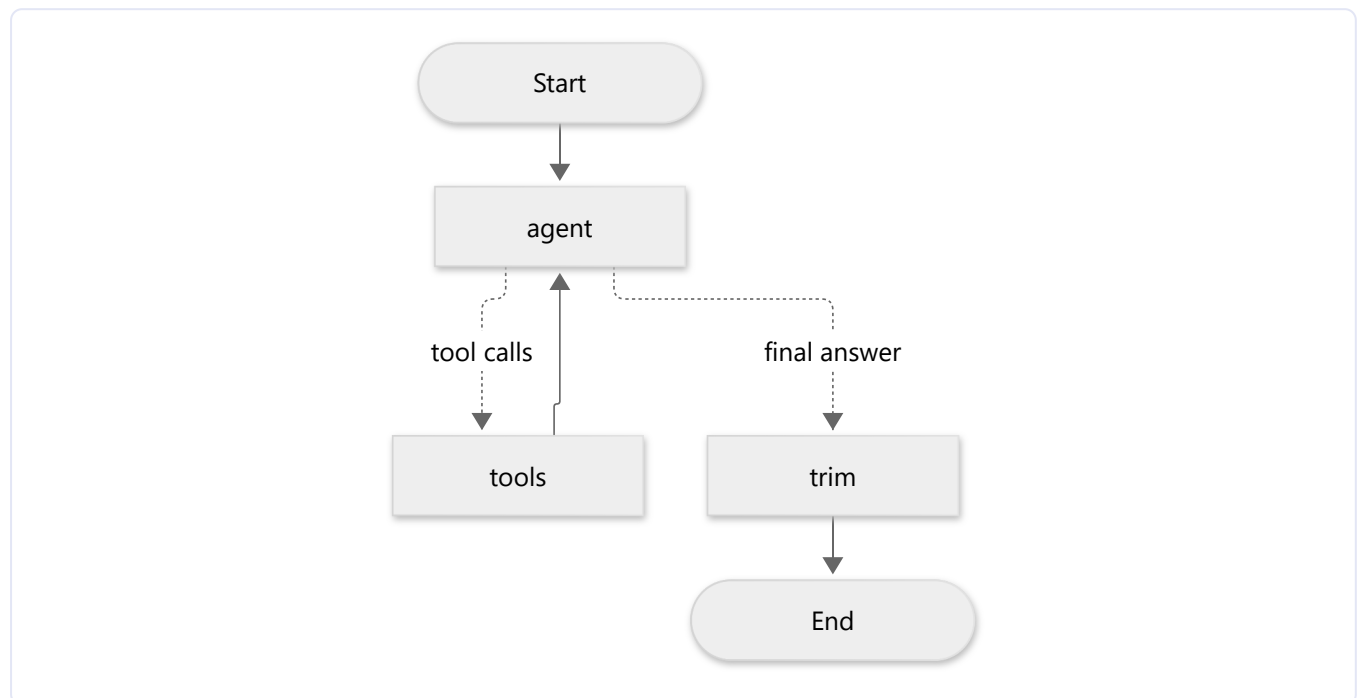


Figure 2. The LangGraph agent graph (printed by `npm run graph`)

- **agent:** calls Claude with a short system prompt (added per call, not stored) and the conversation.

- **tools:** LangGraph's `ToolNode` runs every tool the model asked for, in parallel, and returns errors to the model as messages so it can correct itself (e.g. a bad date).
- **trim:** once a turn is answered, removes its tool calls/results and keeps only the last 4 question/answer pairs. Saves tokens on every later turn.
- **State** = `messages` + `employeeId`. The checkpointer saves state after every step under the thread ID: that is the conversation memory, and it is what lets a run pause and resume.

Tools

Tool	Purpose
<code>get_employee_profile</code>	Grade, department, location, joining date, tenure, manager
<code>get_leave_balance</code>	Allocated, carried forward, used, pending, available per leave type
<code>get_leave_history</code>	The user's requests in a date range
<code>get_holidays</code>	Holiday calendar for a year and office
<code>calculate_leave</code>	Working days in a range (weekends + office holidays excluded), loss of pay, balance after, pro-rata entitlement, encashment
<code>check_eligibility</code>	Tenure and balance check for a leave type
<code>search_policy</code>	Keyword search over the policy documents; returns sections with citation labels
<code>apply_leave</code>	Validates, asks the user to confirm (interrupt), then writes a pending request
<code>cancel_leave</code>	Validates (own request, pending/approved, not started), asks to confirm, then cancels
<i>Manager only</i>	
<code>get_pending_approvals</code>	Direct reports' requests waiting for a decision
<code>get_team_leave</code>	Direct reports' approved and pending leave in a date range
<code>decide_leave_request</code>	Approve or reject a direct report's pending request, after confirmation

No tool takes an employee ID. Every tool reads the identity from graph state, which only the server sets. The three manager tools are bound to the model **only when the logged-in user is a manager** (employees don't pay tokens for them), every manager query is scoped to direct reports in SQL, and each manager tool re-checks the role, so an employee session can't use them even if the model tried.

Applying for leave (human in the loop)

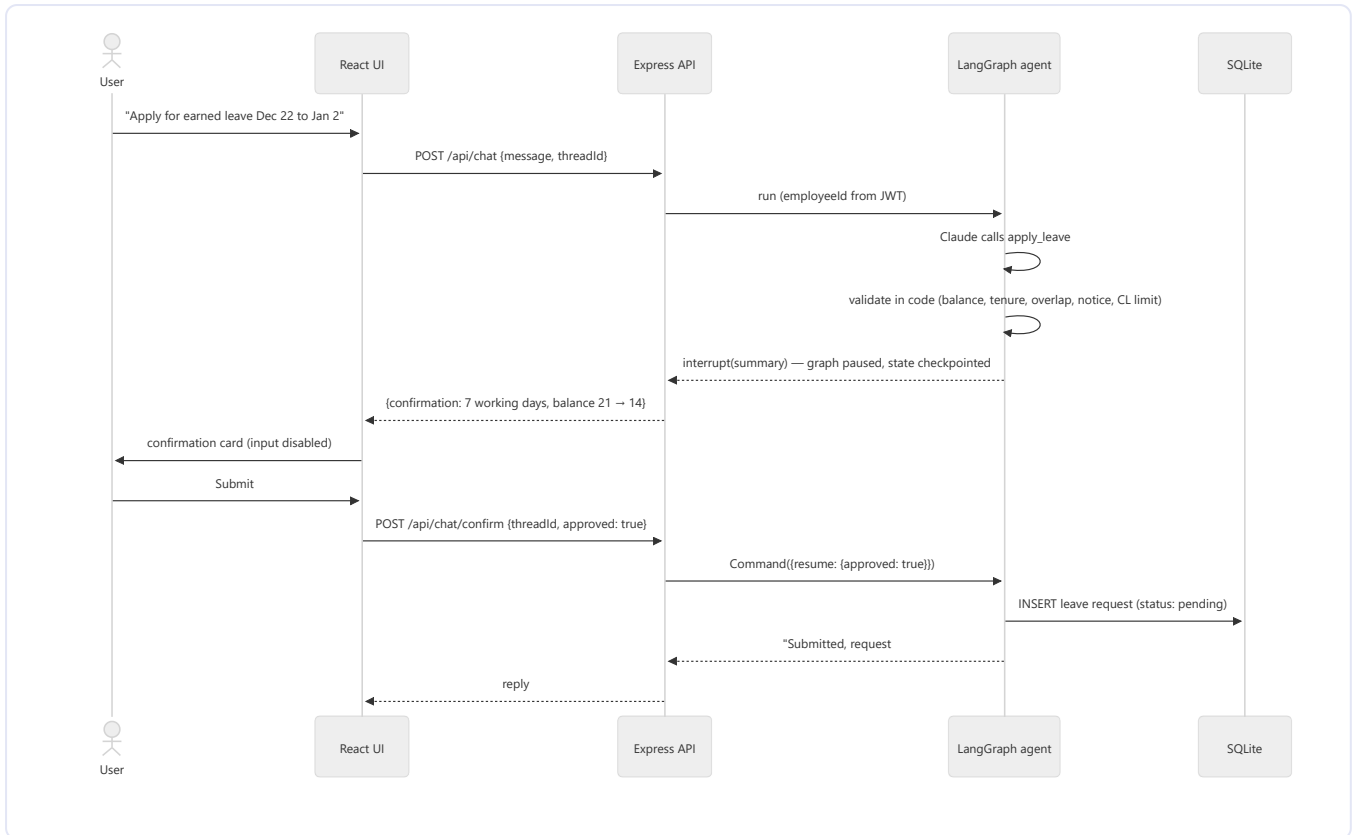


Figure 3. Applying for leave with human-in-the-loop confirmation

If the user cancels, nothing is written. If the request breaks a rule, the agent explains the problem and never shows a confirmation. Cancelling a request and a manager's approve/reject use the same pattern: validate in code, interrupt, write only after confirmation.

5 Design decisions

Math and rules live in code, not in the LLM. Working days, balances, loss of pay, pro-rata and encashment are pure functions with unit tests. The model only chooses tools, fills arguments and explains results; the prompt tells it not to do arithmetic (not even totals).

Identity can't be talked around. `employeeId` flows *verified JWT* → *request* → *graph state* → *tools*. Nothing the user types or sends in the request body can change it; requests for other employees are refused. Conversation threads are keyed `employeeId:threadId`, so a guessed thread ID can't load someone else's chat.

Writes are validated, then confirmed. `apply_leave` checks every hard rule in code (balance, 6-month tenure for earned leave, overlapping requests, no past dates, casual leave ≤ 3 consecutive days, 7 days' notice for 3+ days of earned leave) *before* asking. The LangGraph interrupt pauses the graph; on resume the tool re-runs from the top, so everything before the interrupt is side-effect free and the database write comes last.

Grounded policy answers. Policies are split into short `##` sections and indexed with BM25 at startup. Results must match at least one distinctive word, so a question about an uncovered topic returns nothing and the agent says the policy doesn't cover it. Answers cite the source section.

Keyword search instead of embeddings. Anthropic has no embeddings API, and the policy set is small and well structured. Local BM25 needs no second provider, no API calls and no tokens, and it is deterministic and testable. A small synonym map (vacation → earned leave, WFH → work from home, ...) covers common paraphrases.

Token efficiency as a design goal. Every request costs tokens, so each choice was made for low token use:

Lever	Choice
Model	Claude Haiku 4.5 (cheapest Claude), swappable via <code>LLM_PROVIDER</code> / <code>LLM_MODEL</code>
Prompt + tool schemas	Terse; ~1.5k tokens per call for an employee (9 tools); managers ~1.85k (12 tools)
Role-based tools	Manager tools and prompt rules sent only for managers
Tool results	Compact JSON, no echoed inputs, empty fields omitted
History	Tool traffic removed after each turn; last 4 Q&A pairs kept
Answers	"Under 80 words, table only for 3+ rows" — output tokens cost 5× input
Redundant calls	Prompt and tool descriptions steer to the minimal tool set
Retrieval	Local BM25: zero tokens to search, ≤ 3 short sections returned

Measured examples: a refusal costs 1 model call (~1.7k input tokens); "what's my balance" 2 calls (~3.6k in / ~115 out); a full leave application ~4.2k in / ~230 out — roughly half a US cent per question.

Provider-agnostic. The app only sees LangChain's `BaseChatModel`. Switching to Gemini is a `.env` change; another provider is one `case` in `server/src/llm.ts`. Gemini's free-tier rate limit (5 requests/minute) needed a custom retry handler that honours the server's `retryDelay`; it is included and tested.

6 Security and data

- Synthetic data only; passwords are bcrypt hashes; the seed prints the demo password.
- JWT (HS256, 8h, `algorithms` pinned so `alg: none` is rejected). Wrong password and unknown email return the same message and take the same time.
- Login rate limiting: after 5 failed attempts for the same email and IP within 15 minutes, logins return 429 with `Retry-After` (in memory; a shared store would be needed across several servers).
- Managers see only their direct reports; nobody can decide their own request.
- API returns only public employee fields; malformed or oversized requests are rejected (1000-char messages, 10 kB bodies); server errors are generic to the client.
- The UI keeps the token in memory only (no `localStorage`).
- Secrets live in `server/.env` (gitignored); `.env.example` has placeholders.

7 Testing

```
npm test      # 45 tests, no LLM calls, no API cost
npm run eval # 13 cases against the real model (~$0.07 per run)
```

Area	What is covered
Leave math	Working days across Christmas/New Year, tenure, pro-rata, encashment, loss of pay, invalid dates
Leave validation	Every apply_leave and cancel_leave rule, on a fresh temporary database
Policy search	Section parsing, stemming, correct top section for 6 questions, "not covered" cases
Agent graph	The real graph with a scripted fake model and a temp database: pause → confirm → write, cancel, rule violations, per-employee threads, history cap, manager approve/reject, manager can't decide own request, employee can't use manager tools
API + auth	Login, forged/expired/ alg:none /unknown-user tokens, body employeeId ignored, thread IDs, confirm endpoint (409 when nothing is pending), login rate limiting
Retry handler	Gemini per-minute vs daily quota errors

Evaluation set (`npm run eval`): unit tests check our code with a fake model; the eval checks the real agent. 13 scripted cases (policy citation, "not covered", balance, multi-tool dates, follow-up, apply/cancel/approve with confirmation, rule enforcement, privacy, new joiner, manager flows) assert which tools were called, that no redundant tools were called, which confirmation appeared, and what the answer must or must not contain. It runs on a fresh temporary database with a pinned date and prints pass/fail, tokens and estimated cost. Current result: 13/13. The eval already found and fixed one inefficiency (redundant balance and holiday lookups on date questions).

LangSmith traces (`LANGSMITH_TRACING=true`) show each turn's agent → tools → agent → trim steps, the exact prompts, tool inputs/outputs and token counts.

8 Project structure

```
├─ package.json          # root scripts: setup, dev (API + UI), test, eval, typecheck, db:reset
├─ scripts/free-ports.cjs # runs before dev: stops this project's leftover dev servers
├─ server/
│   ├── policies/        # synthetic policy documents (Leave, Holiday, Attendance/WFH)
│   └── src/
│       ├── agent/        # graph.ts (StateGraph), state.ts, prompt.ts, run.ts (turn runner)
│       ├── tools/        # index.ts (employee tools), manager.ts (team tools), common.ts
│       ├── hr/           # leaveMath (pure), leaveRequest (validation), repo (SQL), dates
│       ├── rag/          # policyIndex.ts (BM25)
│       ├── eval/         # cases.ts + run.ts: evaluation against the real model
│       ├── auth/         # login, JWT sign/verify, login rate limiter
│       ├── server/       # Express app + entry point
│       ├── db/           # schema, seed, temp-DB helper for tests
│       ├── llm.ts        # model factory (provider switch, Gemini retry handler)
│       ├── config.ts     # all settings
│       └── cli.ts        # terminal chat for development
├─ client/src/           # React + MUI: LoginPage, ChatPage, ConfirmLeaveCard, api.ts
└─ docs/PROGRESS.md      # stage-by-stage build log and decision record
```

9 Limitations and future enhancements

- **Persistent memory:** conversations use an in-memory checkpointer and reset when the server restarts. A SQLite/Postgres checkpointer would persist them (the SQLite package currently pins an older better-sqlite3).
- **Semantic retrieval:** embeddings or hybrid search once the policy set grows beyond keyword search's reach.
- **Streaming replies** token by token to the UI (needs to coexist with confirmation pauses).
- **Sessions:** httpOnly cookie + refresh tokens instead of an in-memory bearer token; shared store for rate limits.
- **Leave edge cases:** half-day leave, year-boundary balances (a Dec–Jan request is currently checked against the current year's balance), manager comments on decisions, notifications.
- **Manager scope:** skip-level managers and HR admin views.
- **Prompt caching** once the cached prefix reaches Haiku 4.5's 4,096-token minimum (or on a model with a lower minimum).
- **Eval in CI** with a small per-run budget, plus more cases (paraphrases, multi-turn edge cases).